Facultad de Ingeniería - UBA

Taller de Programación

TA045 - Cátedra Deymonnaz

Proyecto RoomRTC

2do Cuatrimestre - 2025

Grupo: Los Compiladores Felices

Entrega Final: Lunes, 1 de diciembre de 2025

Padrón	**Nombre**	**Email**
111753	Cortez Aguilar, Diego Alejandro	dcortez@fi.uba.ar
111834	Pérez D'Angelo, Tomás	tperezd@fi.uba.ar
112018	Molina Buitrago, Marlon Stiven	mmolinab@fi.uba.ar
112034	Politti, Ignacio	ipolitti@fi.uba.ar

Contents

1 Introducción **3**

1.1 Objetivos del Proyecto 3

1.2 Alcance y Desafíos 3

2 Arquitectura del Sistema **5**

2.1 Componentes Principales 5

2.2 Modelo de Concurrencia 5

3 Implementación **6**

3.1 ICE - Interactive Connectivity Establishment 6

3.1.1 Representación de Candidatos y Priorización 6

3.1.2 Formación de Pares y Verificación 7

3.1.3 Selección de Conectividad 7

3.2 SDP - Session Description Protocol 8

3.2.1 Estructura de la Negociación 8

3.2.2 Flujo de Señalización 8

3.3 RTP - Real-time Transport Protocol 9

3.3.1 Estructura del Paquete y Serialización 10

3.3.2 Arquitectura del Emisor (`RtpSender`) 11

3.3.3 Arquitectura del Receptor (`RtpReceiver`) 11

3.3.4 Jitter Buffer 12

3.4 RTCP - Real-time Transport Control Protocol 14

3.4.1 Establecimiento de Conexión 14

3.4.2 Estructura de Paquetes y Serialización 15

3.4.3 Mecanismo de Reporte y Estadísticas 15

3.5 Seguridad en el Transporte: DTLS 16

3.5.1 Fase 1: Identidad y Señalización (Pre-Handshake) 16

3.5.2 Fase 2: Negociación y Verificación (Handshake) 16

3.5.3 Fase 3: Exportación de Material Clave (RFC 5764) 17
3.6 Servidor Central y Señalización 18
3.6.1 Gestión de Estado y Persistencia 18
3.6.2 Protocolo Cliente-Servidor 18

4 Instrucciones de Compilación y Ejecución 20
4.1 Compilación 20
4.2 Ejecución 20
4.2.1 Servidor 20
4.2.2 Cliente 20
4.3 Testing 20
4.4 Documentación 21

5 Bibliografía 22

1 Introducción

Desde la pandemia de COVID-19 en 2020, el software de videoconferencias ha cobrado una importancia central en la infraestructura global, permitiendo la continuidad de clases, conferencias y eventos de manera remota. En este contexto, la Facultad de Ingeniería de la Universidad de Buenos Aires propuso el desafío de desarrollar una solución propia que cumpla con altos estándares de usabilidad, escalabilidad y eficiencia.

El presente trabajo, titulado **RoomRTC**, consiste en el diseño e implementación de un sistema de videoconferencias desarrollado íntegramente en el lenguaje de programación **Rust**. La elección de este lenguaje responde a la necesidad de garantizar seguridad de memoria (*memory safety*) y alto rendimiento sin sacrificar la ergonomía del desarrollo, características críticas para aplicaciones de tiempo real.

1.1 Objetivos del Proyecto

El objetivo principal de este trabajo práctico es desarrollar una implementación propia de la pila de protocolos **WebRTC** (*Web Real-Time Communication*). A diferencia de soluciones comerciales que utilizan librerías preexistentes para la gestión de la red, este proyecto requirió la implementación manual de los protocolos fundamentales que permiten la comunicación punto a punto (*Peer-to-Peer*) entre dispositivos.

Los objetivos específicos incluyen:

- **Implementación de Protocolos de Red:** Desarrollo desde cero de mecanismos para el establecimiento de sesiones (SDP), la conectividad a través de NAT (ICE), el transporte de medios en tiempo real (RTP/RTCP) y la seguridad en el transporte (DTLS).
- **Arquitectura Cliente-Servidor y P2P:** Diseño de un servidor central de señalización para el descubrimiento de pares y una arquitectura distribuida para la transmisión de video.
- **Procesamiento Multimedia:** Captura, codificación (H.264) y decodificación de video en tiempo real, gestionando la latencia y la posible pérdida de paquetes inherente al protocolo UDP.
- **Aplicación de Buenas Prácticas:** Uso de herramientas de integración continua, manejo idiomático de errores en Rust, concurrencia segura y documentación técnica exhaustiva.

1.2 Alcance y Desafíos

El sistema RoomRTC permite a los usuarios registrarse, visualizar usuarios conectados en tiempo real e iniciar videollamadas. El mayor desafío técnico del proyecto residió en la orquestación de múltiples hilos de ejecución para manejar la interfaz gráfica, la red y

el procesamiento de video de forma concurrente sin bloqueos (*busy-wait*) ni condiciones de carrera.

Asimismo, se puso especial énfasis en la robustez del sistema frente a condiciones de red adversas, implementando mecanismos como *Jitter Buffers* para el reordenamiento de paquetes y algoritmos de re-sincronización de video ante la pérdida de cuadros clave (*Keyframes*).

A lo largo de este informe se detallan las decisiones de diseño tomadas, la arquitectura de los componentes implementados y el análisis de los protocolos que hacen posible la comunicación en tiempo real.

2 Arquitectura del Sistema

El sistema RoomRTC sigue una arquitectura híbrida Cliente-Servidor para la señalización y Peer-to-Peer (P2P) para la transmisión de medios.

2.1 Componentes Principales

- **Servidor Central (Signaling Server):** Es el punto de encuentro. Su responsabilidad es la gestión de usuarios (registro, login), el mantenimiento del estado (quién está conectado/ocupado) y el enrutamiento de mensajes de señalización (SDP) entre pares. Se comunica vía TCP/TLS.
- **Cliente (Peer):** La aplicación de usuario final. Maneja la captura de video, la interfaz gráfica, la codificación/decodificación y la transmisión P2P vía UDP.

2.2 Modelo de Concurrencia

Dado que Rust garantiza la seguridad de memoria en tiempo de compilación, se optó por un modelo de concurrencia basado en el paso de mensajes (*Message Passing*) en lugar de memoria compartida. La aplicación cliente utiliza múltiples hilos (*threads*) dedicados:

1. **Hilo UI (Main):** Maneja el renderizado de la interfaz gráfica (eframe/egui).
2. **Hilo de Red (Controller):** Gestiona la conexión TCP con el servidor y la lógica de negocio.
3. **Pipeline de Medios:** Hilos independientes para captura (Cámara), codificación, envío RTP, recepción RTP y decodificación.

La comunicación entre estos hilos se realiza mediante canales `mpsc` (Multi-producer, single-consumer), garantizando que el bloqueo de la interfaz gráfica o de la red no interrumpa el procesamiento de video en tiempo real.

3 Implementación

3.1 ICE - Interactive Connectivity Establishment

El establecimiento de conectividad en RoomRTC se basa en una implementación parcial del estándar ICE (RFC 8445). Este módulo tiene como objetivo descubrir, validar y seleccionar la ruta de red más eficiente para la transmisión de medios (RTP/UDP) entre dos pares.

La arquitectura del módulo se centra en el IceAgent, un orquestador que gestiona el ciclo de vida de la conexión a través de tres fases principales: recolección de candidatos, formación de pares y verificaciones de conectividad.

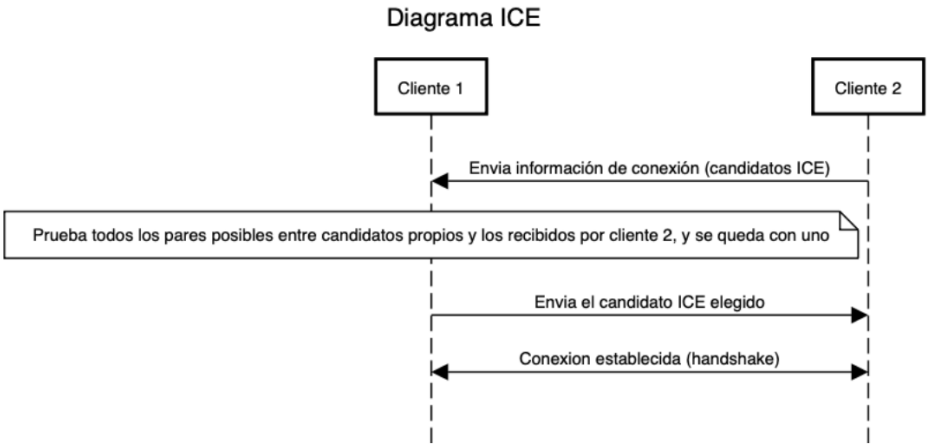

Figure 1: Diagrama ICE

3.1.1 Representación de Candidatos y Priorización

La unidad fundamental de conectividad es el `Candidate`. Cada candidato representa un punto final de transporte potencial, caracterizado por una dirección IP, un puerto y un protocolo de transporte (UDP). La implementación soporta dos tipos de candidatos:

- **Host Candidate:** Representa la interfaz de red física del dispositivo. Se obtiene enumerando las interfaces de red locales (excluyendo *loopback*) mediante llamadas al sistema operativo.
- **Server Reflexive Candidate (srflx):** Representa la dirección pública del par vista desde Internet. Esta se obtiene mediante un cliente STUN integrado que realiza transacciones de *Binding Request* contra servidores públicos (como Google STUN), permitiendo al agente descubrir su mapeo NAT externo.

Para determinar qué candidato utilizar, el sistema implementa la fórmula de cálculo de prioridad estipulada en el RFC 8445. La prioridad es un entero de 32 bits que combina

la preferencia del tipo de candidato (*type_preference*), la preferencia local y el ID del componente:

$$Priority = (2^{24} \times TypePreference) + (2^{8} \times LocalPreference) + (256 - ComponentID)$$

En nuestra implementación, se asigna una preferencia mayor a los candidatos *Host* (126) sobre los *Server Reflexive* (100), priorizando siempre la conexión directa en red local (LAN) si está disponible, ya que ofrece menor latencia y mayor ancho de banda.

3.1.2 Formación de Pares y Verificación

Una vez que el agente local ha recolectado sus candidatos y ha recibido los candidatos del par remoto (vía señalización SDP), se procede a la fase de emparejamiento. El sistema genera una lista de `CandidatePair`, que representa el producto cartesiano entre todos los candidatos locales y remotos disponibles.

Cada par tiene asociada una prioridad de par única, calculada matemáticamente para garantizar que ambos agentes (el local y el remoto) coincidan en el orden de preferencia de los pares a verificar. La fórmula implementada favorece a los pares formados por candidatos de alta prioridad individual:

$$PairPriority = 2^{32} \times \min(G, D) + 2 \times \max(G, D) + (G > D ? 1 : 0)$$

Donde G y D son las prioridades del candidato controlador y controlado respectivamente.

3.1.3 Selección de Conectividad

Finalmente, el `IceAgent` ejecuta el proceso de verificación de conectividad de manera secuencial y priorizada.

El agente ordena la lista de pares según su prioridad y los evalúa uno por uno. Para validar la viabilidad de un par, se utiliza un mecanismo de *handshake* bidireccional simplificado, inspirado en los *Binding Requests* del estándar ICE (RFC 8445):

1. El agente local envía un mensaje de prueba ("PING") a la dirección candidata remota.
2. Se inicia una espera bloqueante con un tiempo límite (*timeout*) reducido.
3. Si se recibe una respuesta ("PING") desde la dirección esperada, se confirma la ruta de salida.
4. Simultáneamente, si el agente recibe un "PING" del par remoto durante este proceso, responde inmediatamente con un "PONG", garantizando la validación en ambas direcciones (*Triggered Checks*).

El primer par que complete este intercambio exitosamente es marcado como `Succeeded` y el proceso de búsqueda se detiene inmediatamente. Este par seleccionado (`SelectedPair`) define la dirección IP y puerto definitivos hacia los cuales se dirigirán los flujos de video RTP y los mensajes de control RTCP.

3.2 SDP - Session Description Protocol

Para coordinar los distintos módulos de comunicación (ICE, DTLS y RTP), el sistema utiliza el protocolo SDP (RFC 4566) bajo el modelo de negociación *Offer/Answer* (RFC 3264). SDP no transporta medios por sí mismo, sino que actúa como un formato de descripción estandarizado que se intercambia a través de un servidor de señalización (vía WebSocket/HTTP) antes de establecer la conexión *Peer-to-Peer*.

La responsabilidad de este módulo es serializar el estado local del agente en un mensaje de texto y deserializar la descripción del par remoto para configurar los decodificadores y transportes.

3.2.1 Estructura de la Negociación

En RoomRTC, un mensaje SDP cumple tres funciones críticas mediante líneas de atributos específicas:

- **Negociación de Medios (`m=`):** Define el tipo de contenido (video), el puerto base, el protocolo de transporte (UDP/TLS/RTP/SAVPF) y el formato de codificación (H.264). Esto permite que ambos pares acuerden un códec común antes de iniciar la transmisión.
- **Intercambio de Candidatos ICE (`a=candidate`):** Cada candidato descubierto por el *IceAgent* (Host o Server Reflexive) se serializa en una línea de atributo. Esto permite que el par remoto conozca las direcciones IP y puertos disponibles para intentar la conexión.
- **Identidad Criptográfica (`a=fingerprint`):** Transporta el hash SHA-256 del certificado DTLS local. Como se detalló en la sección de seguridad, este atributo es vital para autenticar el handshake DTLS y prevenir ataques *Man-in-the-Middle*.

3.2.2 Flujo de Señalización

El proceso de establecimiento de sesión sigue un flujo asincrónico estricto:

1. **Generación de Oferta (Offer):** El par iniciador recolecta sus candidatos ICE, genera sus claves DTLS y crea un documento SDP que describe estas capacidades.
2. **Intercambio:** La oferta se envía al servidor de señalización, el cual la enruta hacia el par destino.

3. **Aplicación y Respuesta (Answer):** El par receptor aplica la oferta remota (configurando su stack de red) y genera una respuesta SDP con sus propios candidatos y fingerprint, cerrando el ciclo de negociación.

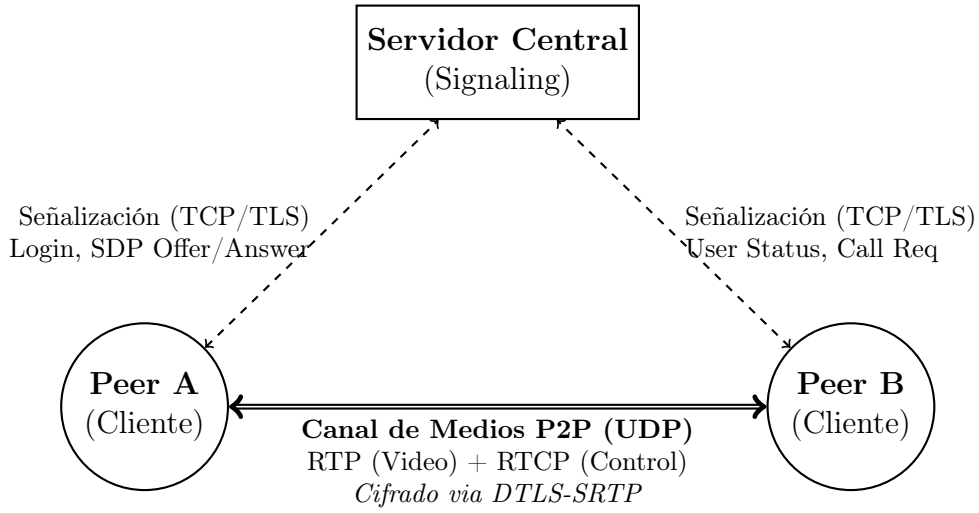

Figure 2: Topología de red de RoomRTC: Señalización centralizada y medios distribuidos.

3.3 RTP - Real-time Transport Protocol

El núcleo de la transmisión de video en RoomRTC es una implementación del protocolo RTP sobre UDP. El diseño delega la reconstrucción temporal y la integridad de los cuadros de video a la lógica de la aplicación.

El módulo RTP no solo encapsula los datos, sino que actúa como gestor de estado del flujo de video, coordinando la fragmentación en el origen y el reordenamiento en el destino mediante un *Jitter Buffer*.

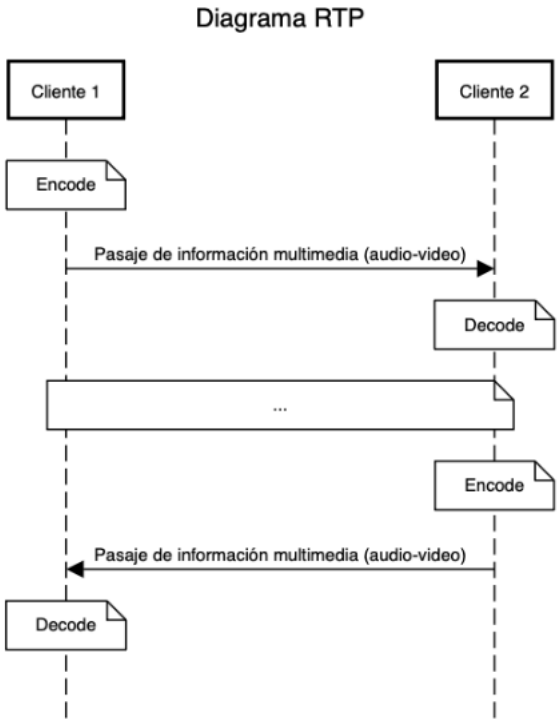

Figure 3: Diagrama RTP

3.3.1 Estructura del Paquete y Serialización

Se diseñó la estructura `RtpPacket` con un esquema de serialización binaria. Dado que los cuadros de video comprimidos (H.264) frecuentemente superan el MTU (*Maximum Transmission Unit*) de la red, el diseño del encabezado incluye extensiones específicas para gestionar la fragmentación y el reensamblado de estas unidades.

El paquete se compone de los siguientes campos:

- **Version (V)**: 2 bits. Establecida en 2, cumpliendo con el estándar.
- **Payload Type (PT)**: 8 bits. Identifica el formato de codificación del video (e.g., H.264).
- **Sequence Number**: 16 bits. Un contador incremental por cada paquete enviado. Este campo es crítico para que el receptor pueda posicionar el fragmento de video en la línea de tiempo correcta y detectar pérdidas.
- **Timestamp:** 32 bits. Marca de tiempo del muestreo del cuadro de video. Todos los paquetes que constituyen un mismo cuadro (frame) comparten este valor, permitiendo su agrupación temporal.
- **SSRC**: 32 bits. Identificador aleatorio de la fuente de sincronización.
- **Marker**: 1 bit. Se utiliza como bandera de finalización de cuadro, indicando que el paquete actual contiene el último fragmento de la imagen.

- **Extensiones de Fragmentación** (`frame_id, chunk_id`): Se añadieron identificadores de 64 bits para vincular inequívocamente los fragmentos a un cuadro específico y su orden relativo, robusteciendo la reconstrucción frente a reinicios del contador de secuencia.

3.3.2 Arquitectura del Emisor (`RtpSender`)

El componente `RtpSender` gestiona la emisión del flujo de video. Su funcionamiento interno se basa en la gestión de estado de la secuencia:

- **Gestión de Secuencia**: Mantiene un contador interno (`next_sequence_number`) que se incrementa con cada datagrama emitido. La implementación maneja explícitamente el desbordamiento (*wrapping*) del entero de 16 bits, asegurando la continuidad lógica del flujo.
- **Fragmentación**: Recibe los fragmentos de video pre-procesados por el codificador y les asigna los metadatos RTP correspondientes antes de la transmisión al socket UDP.
- **Estadísticas**: Actualiza contadores atómicos de paquetes y octetos transmitidos. Estos contadores son la fuente de verdad para el módulo RTCP, permitiendo la generación de reportes de sender (`SenderReport`)

3.3.3 Arquitectura del Receptor (`RtpReceiver`)

El componente `RtpReceiver` actúa como la puerta de entrada para el flujo multimedia, siendo responsable de la ingesta, validación y normalización de los datos provenientes de la red antes de que estos sean procesables por el decodificador. Su funcionamiento se basa en un ciclo de procesamiento continuo que transforma datagramas crudos en estructuras de datos coherentes.

Sus funciones principales son:

1. **Adaptación de Protocolo**: Lee los datagramas UDP y realiza la deserialización binaria según la especificación RTP definida en el proyecto. Transforma flujos de bytes desestructurados en objetos `RtpPacket` con campos tipados (secuencia, timestamp, payload).
2. **Validación de Integridad**: Actúa como primer filtro de seguridad, descartando paquetes mal formados, con versiones de protocolo incorrectas o aquellos que no cumplen con la longitud mínima de encabezado, protegiendo a las capas superiores de datos corruptos.
3. **Gestión de Errores de Red**: Monitorea el estado del socket subyacente. Ante errores críticos de recepción (distintos a un simple timeout de lectura), tiene la capacidad de señalar el cierre de la sesión al manejador de reportes (RTCP), deteniendo la conexión de manera controlada.

4. **Interacción con el Buffer**: Su diseño desacopla la llegada de datos de su consumo. En lugar de entregar los paquetes directamente a la capa de video, los inyecta inmediatamente en el Jitter Buffer.

La lógica de entrega de paquetes (`receive`) prioriza la continuidad del video: siempre verifica primero si el Jitter Buffer tiene un paquete listo para ser reproducido (ordenado y a tiempo).

3.3.4 Jitter Buffer

El Jitter Buffer es el encargado de transformar un flujo de paquetes UDP desordenado y con tiempos de llegada variables en un flujo de video continuo y ordenado.

Para el almacenamiento y ordenamiento se utiliza internamente árbol binario (`BTreeMap<u16, RtpPacket>`), que garantiza que los paquetes se almacenen siempre ordenados por su número de secuencia, independientemente del orden aleatorio de llegada desde la red, facilitando una extracción secuencial eficiente.

La extracción de paquetes sigue una máquina de estados estricta:

1. **Verificación de secuencia**: se intenta extraer estrictamente el paquete siguiente al último reproducido (`last_played_seq_num + 1`), manejando la aritmética modular para los saltos de secuencia.
2. **Playout Delay**: se aplica un retardo configurable (`playout_delay`) comparando el tiempo de llegada del paquete con el instante actual. Esto crea una ventana de tiempo que permite la llegada de paquetes retrasados (out-of-order) antes de que sean necesarios para la decodificación.
3. **Descarte por obsolescencia**: si un paquete permanece en el buffer más allá de un umbral máximo (`MAX_JITTER_WAIT_MILLIS`), se considera obsoleto para video en tiempo real y se descarta, contabilizándolo como pérdida para evitar congelar la imagen indefinidamente.

Para la estimación del Jitter según RFC 3550, el buffer calcula la varianza en los tiempos de llegada (Jitter) para retroalimentar al emisor. Se implementó la fórmula:

$$J(i) = J(i-1) + \frac{|D(i-1,i)| - J(i-1)}{16}$$

Donde $D(i-1,i)$ representa la diferencia entre el tiempo de llegada relativo y el timestamp RTP de dos paquetes consecutivos. Esta métrica es fundamental para que el sistema monitoree la estabilidad de la conexión de video.

El Jitter Buffer no solo ordena paquetes, sino que implementa lógica específica para manejar la naturaleza predictiva del códec H.264:

- **Dependencia de I-Frames:** Si el buffer detecta una discontinuidad grave o un vaciado total (*buffer underrun*), entra en estado de `i_frame_needed`. En este estado, descarta todos los paquetes P-Frame (deltas) entrantes hasta recibir un nuevo I-Frame (Keyframe), evitando mostrar artefactos visuales corruptos.
- **Buffer Circular:** Se implementa sobre un array de tamaño fijo para evitar asignaciones de memoria dinámicas constantes.
- **Tolerancia Temporal:** Se valida el *timestamp* de los paquetes antes de entregarlos al decodificador. Si un cuadro completo llega demasiado tarde para su tiempo de reproducción (*playout time*), se descarta para mantener la latencia baja, priorizando el tiempo real sobre la fluidez absoluta.

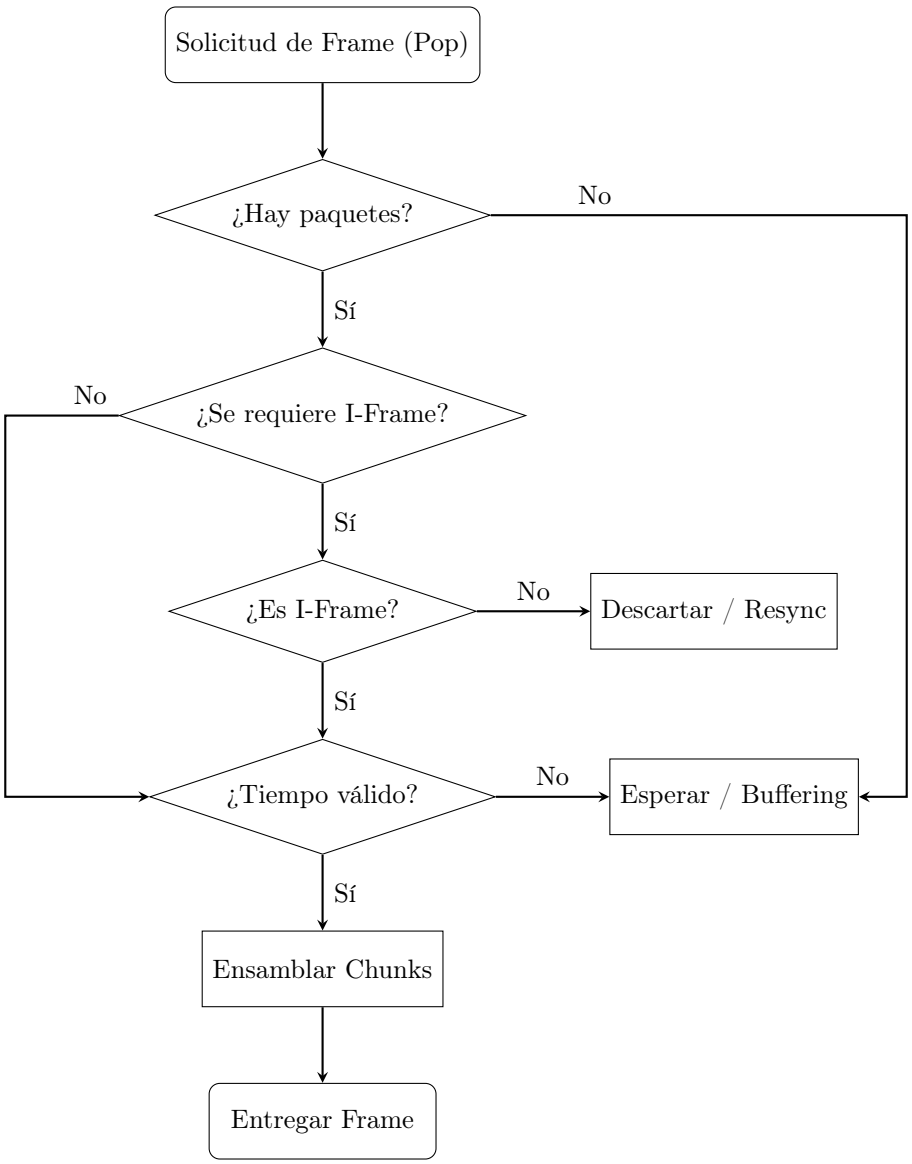

Figure 4: Lógica de extracción y validación del Jitter Buffer.

3.4 RTCP - Real-time Transport Control Protocol

El protocolo de control RTCP funciona en conjunto con RTP. Implementa los reportes estándar definidos en el RFC 3550 y también actúa como el gestor del ciclo de vida de la conexión, administrando el establecimiento inicial (handshake) y la finalización de la llamada.

La implementación se centraliza en el `RtcpReportHandler`, un componente que opera sobre un socket UDP dedicado (típicamente en el puerto $N + 1$ respecto al puerto RTP) y coordina la comunicación de control.

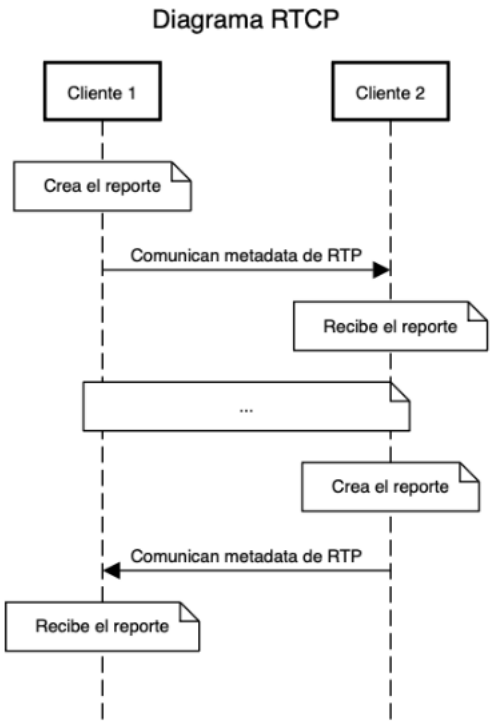

Figure 5: Diagrama RTCP

3.4.1 Establecimiento de Conexión

El sistema implementa un protocolo de handshake activo para verificar la conectividad bidireccional antes de iniciar la transmisión de video pesado.

El flujo de estados implementado es el siguiente:

- Hello: El cliente envía paquetes Hello periódicos y espera una respuesta.
- Ready: Al recibir un Hello del par remoto, responde con un paquete Ready.
- Open: Al recibir un Ready, se confirma que el canal es fiable en ambas direcciones y se transiciona el estado de la conexión a `ConnectionStatus::Open`, habilitando el inicio de los hilos de transmisión de medios.

3.4.2 Estructura de Paquetes y Serialización

Se implementó un esquema de paquetes híbrido que soporta tanto los formatos binarios estándar como mensajes de señalización ligera:

- **Header Común**: Todos los paquetes incluyen un encabezado de 4 bytes con la versión (2), el tipo de paquete (Payload Type) y la longitud.
- **Sender Report (SR - Tipo 200)**: Enviado por los participantes que están transmitiendo video activamente. Incluye contadores de paquetes y octetos enviados, así como marcas de tiempo NTP/RTP para la sincronización.
- **Receiver Report (RR - Tipo 201)**: Enviado para informar sobre la calidad de la recepción. Transporta bloques de reporte (`ReportBlock`) que contienen métricas críticas como la fracción de pérdida y el jitter acumulado.
- **Mensajes de Control**: Se definieron tipos de paquetes personalizados (`Hello`, `Ready`, `ConnectivityReport`) para mantener la sesión activa (keep-alive) cuando no hay datos de video fluyendo o durante la negociación inicial.

3.4.3 Mecanismo de Reporte y Estadísticas

El `RtcpReportHandler` opera mediante dos hilos de ejecución concurrentes que monitorean y reportan el estado del sistema sin bloquear el procesamiento de video.

Hilo Emisor: se ejecuta periódicamente (según `report_period_millis`) y recopila estadísticas accediendo al estado compartido de los módulos RTP:

1. Consulta de Estado: Accede de manera segura a los contadores del `RtpSender` (paquetes enviados) y a las métricas del Jitter Buffer en el `RtpReceiver`.
2. Generación de Reporte:
   - Si el cliente está enviando video, genera un Sender Report (SR).
   - Si solo está recibiendo, genera un Receiver Report (RR) con las estadísticas de pérdida y jitter calculadas por el buffer.
   - Ante la ausencia de flujo de video, se envía un Connectivity Report. Este paquete ligero actúa como un keep-alive, confirmando al receptor que la conexión sigue activa y evitando así un cierre accidental por tiempo de espera.

Hilo Receptor: escucha los reportes entrantes con un tiempo límite configurable (`receive_limit_millis`).

- Si se reciben reportes válidos, se actualiza la fecha de última actividad.
- Si se deja de recibir tráfico de control por un periodo prolongado o se recibe un paquete `Goodbye` (BYE), el receptor asume que el par se ha desconectado o que la red ha fallado, y procede a cerrar controladamente la sesión, deteniendo los hilos de video para liberar recursos.

3.5 Seguridad en el Transporte: DTLS

Para garantizar la confidencialidad, integridad y autenticación de la comunicación multimedia sobre UDP, el sistema implementa el protocolo **DTLS** (Datagram Transport Layer Security) versión 1.2. La implementación utiliza la biblioteca `udp_dtls` como *wrapper* de `OpenSSL`.

A diferencia de TLS sobre TCP, DTLS maneja la posible pérdida y reordenamiento de paquetes inherente a UDP, estableciendo un canal seguro que sirve como precursor obligatorio para el intercambio de medios mediante SRTP. El ciclo de vida de la seguridad se divide en tres fases críticas:

3.5.1 Fase 1: Identidad y Señalización (Pre-Handshake)

Antes de establecer la conexión de red, cada par genera su identidad criptográfica local de manera efímera para la sesión:

1. **Generación de Certificados:** Al iniciar, el cliente utiliza `OpenSSL` para generar un par de claves RSA de 2048 bits y un certificado X.509 autofirmado.
2. **Cálculo de Fingerprint:** Se calcula el hash SHA-256 del certificado generado. Este resumen criptográfico (*fingerprint*) actúa como la identidad pública del par.
3. **Intercambio SDP:** El *fingerprint* se incrusta en el mensaje SDP (Session Description Protocol) bajo el atributo `a=fingerprint`. Esto permite que los pares intercambien sus identidades de confianza a través del servidor de señalización antes de cualquier contacto directo.

3.5.2 Fase 2: Negociación y Verificación (Handshake)

Una vez que ICE ha seleccionado un par de candidatos, se inicia el handshake DTLS sobre los sockets UDP conectados.

- **Roles DTLS (RFC 4145):** Para evitar condiciones de carrera, los roles de cliente y servidor se determinan mediante el atributo SDP `a=setup`. El par con rol *active* inicia la conexión (ClientHello), mientras que el par *passive* o *actpass* espera la conexión.
- **Mitigación de Man-in-the-Middle:** Durante el handshake, el sistema extrae el certificado presentado por el par remoto y calcula su *fingerprint*. La conexión solo se considera válida si este hash coincide exactamente con el valor recibido previamente vía SDP. Si la verificación falla, la conexión se rechaza inmediatamente.

3.5.3 Fase 3: Exportación de Material Clave (RFC 5764)

El túnel DTLS no se utiliza para encapsular el video directamente, sino para negociar las claves de cifrado. Mediante el mecanismo *Keying Material Exporter*:

1. Se invoca la función de exportación con la etiqueta `"EXTRACTOR-dtls_srtp"`.
2. Se generan 60 bytes de material criptográfico seguro.
3. Este material se divide para derivar las *Master Keys* y *Master Salts* (Cliente y Servidor) que alimentan el contexto **SRTP**.

Finalmente, estas claves derivadas permiten cifrar (AES-128-CTR) y autenticar (HMAC-SHA1) cada paquete RTP de video con alto rendimiento, sin la sobrecarga de mantener el túnel DTLS activo para cada frame.

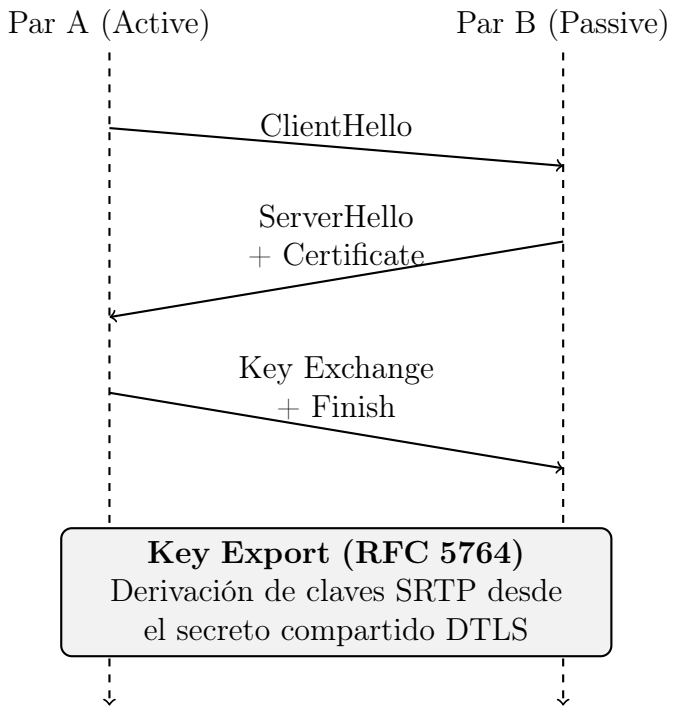

Figure 6: Diagrama del establecimiento de claves DTLS-SRTP.

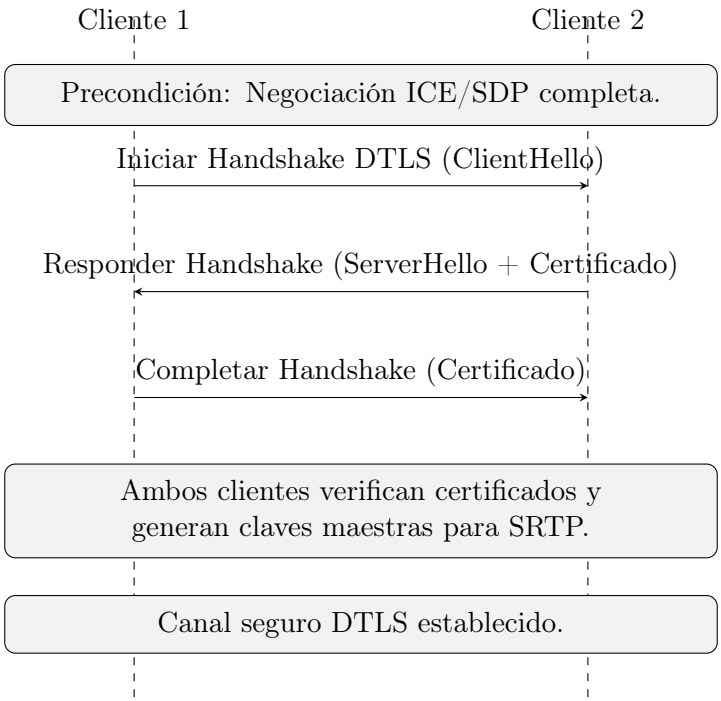

Figure 7: Secuencia de establecimiento DTLS utilizada para derivar claves SRTP.

3.6 Servidor Central y Señalización

El servidor actúa como una entidad de coordinación. Se implementó utilizando `TcpListener` para aceptar conexiones entrantes. Para garantizar la seguridad en la etapa de login y señalización, toda la comunicación TCP está encapsulada en `TLS` (Transport Layer Security) utilizando la biblioteca `rustls` y certificados X.509.

3.6.1 Gestión de Estado y Persistencia

El servidor mantiene en memoria una estructura protegida por `RwLock` que mapea los usuarios conectados a su estado actual (`Available`, `Occupied`, `Offline`) y su flujo de red. La persistencia de las credenciales de usuario se realiza mediante un archivo de texto plano configurado en `room_rtc.conf`, permitiendo que el registro de usuarios sobreviva a reinicios del servidor.

3.6.2 Protocolo Cliente-Servidor

Se diseñó un protocolo de texto simple basado en mensajes delimitados para la comunicación de control. Algunos ejemplos de mensajes son:

- `LOGIN|user|pass`: Autenticación.
- `CALLREQ|token|sdp|target`: Solicitud de llamada retransmitida al destinatario.

- `USERSTATE|user|status`: Notificación de cambio de estado (push) hacia los clientes conectados.

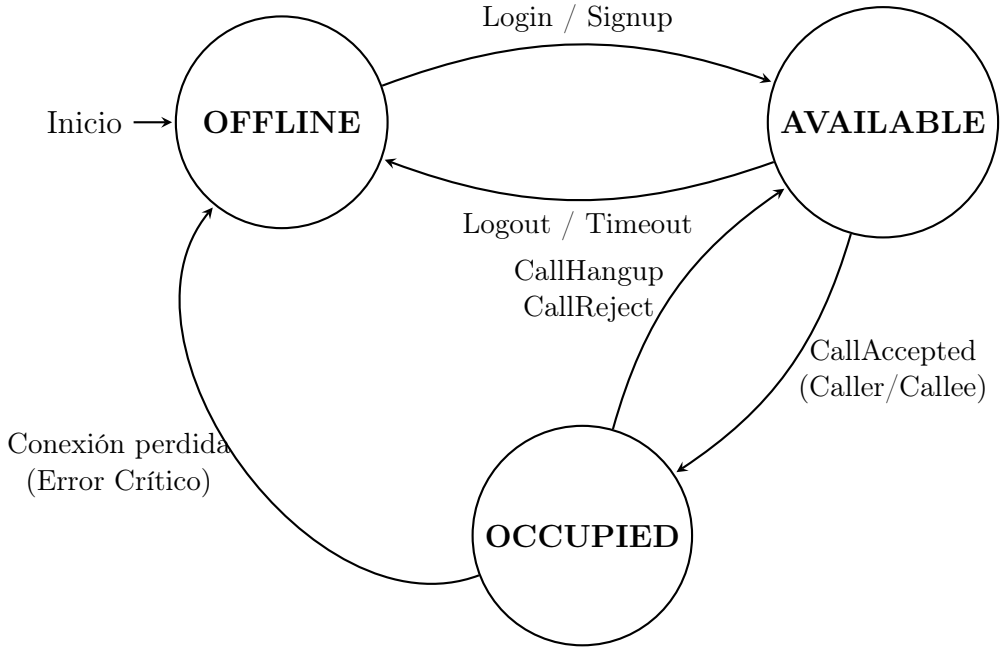

Figure 8: Máquina de estados de usuario en el Servidor Central.

4 Instrucciones de Compilación y Ejecución

A continuación se detallan los pasos necesarios para compilar, ejecutar, probar y generar la documentación del proyecto.

4.1 Compilación

Para compilar el código fuente y descargar las dependencias necesarias, ejecutar el siguiente comando en la raíz del proyecto:

```
cargo build
```

4.2 Ejecución

Importante: Para el correcto funcionamiento del sistema, se debe respetar estrictamente el orden de ejecución: primero debe iniciarse el servidor y posteriormente los clientes.

4.2.1 Servidor

Para iniciar el servidor, ejecute el siguiente comando indicando la ruta al archivo de configuración:

```
cargo run --bin server /path/to/file.conf
```

Tras iniciarse, el servidor imprimirá por salida estándar la dirección (`address`) en la que está escuchando conexiones. Esta dirección es necesaria para el siguiente paso.

4.2.2 Cliente

Para iniciar una instancia de cliente, ejecute el comando indicando el archivo de configuración y la dirección (`address`) obtenida del servidor:

```
cargo run --bin client /path/to/file.conf <address>
```

4.3 Testing

Para ejecutar la suite de pruebas automatizadas (incluyendo la verificación de conectividad ICE y lógica de sesión), utilice:

```
cargo test
```

4.4 Documentación

Para generar la documentación interna del código a partir de los comentarios y abrirla automáticamente en el navegador:

```
cargo doc --open
```

5 Bibliografía

References

[1] Lele, Amogh. Dyte. *WebRTC 102: Demystifying ICE*. Recuperado de https://dyte.io/blog/webrtc-102-demystifying-ice/

[2] Handley, M., Jacobson, V., & Perkins, C. (2006). *SDP: Session Description Protocol* (RFC 4566). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc4566

[3] Keranen, A., Holmberg, C., & Rosenberg, J. (2018). *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal* (RFC 8445). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc8445

[4] Klabnik, S., & Nichols, C. (2018). *The Rust Programming Language*.

[5] Makeitnow. (s.f.). *Getting started with opencv on Rust* [Video]. YouTube.

[6] McGrew, D., & Rescorla, E. (2010). *DTLS Extension to Establish Keys for the SRTP (DTLS-SRTP)* (RFC 5764). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc5764

[7] The Rust Team. (s.f.). *Struct std::net::UdpSocket*. Rust Documentation. Recuperado de https://doc.rust-lang.org/std/net/struct.UdpSocket.html

[8] Schulzrinne, H., Casner, S., Frederick, R., & Jacobson, V. (2003). *RTP: A Transport Protocol for Real-Time Applications* (RFC 3550). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc3550

[9] The WebRTC Series. (s.f.). *WebRTC Architecture (The WebRTC Series — Part 1)*. Medium.

[10] WebRTC for the Curious. (s.f.). *WebRTC for the Curious: What, Why and How*. Recuperado de https://webrtcforthecurious.com/

[11] WebRTC Project. (s.f.). *WebRTC: An open framework for the web that enables Real-Time Communications (RTC) capabilities in the browser*. Recuperado de https://webrtc.org/